

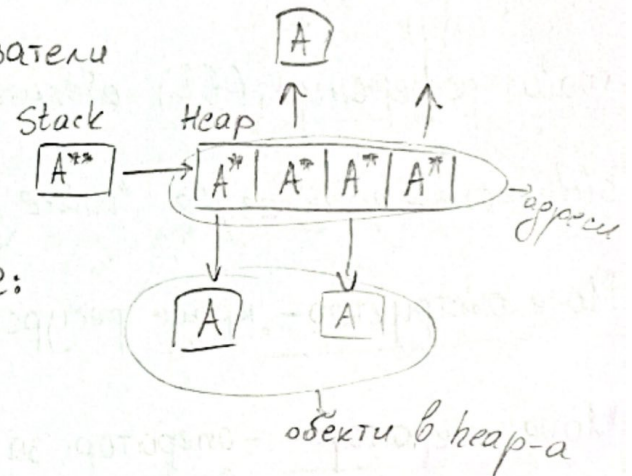
Масиви от обекти и Масиви от указатели в C++

- Масив от обекти - наредена колекция от обекти от един тип, наредени един след друг в паметта, всеки елемент е пълноценен обект и отговаря за живота си
- Масив от указатели към обекти - съдържа адреси на обекти, а не самите обекти
обекта се създава и унищожават ръчно

динамичен масив от указатели:

```
A** arr = new A*[n];      → масив от указатели
for (int i=0; i<n; i++)
{ arr[i] = new Student(...); }

for (int i=0; i<n; i++)    ← освобождаване:
    delete arr[i];        ← 1. всеки обект
delete[] arr;              ← 2. масива
```



Предимства пред масив от обекти:

- Не изисква конструктор по подразбиране
- Указател може да е nullptr, не всяка позиция е заета
- при sort - размястват се указатели, обектите не се копират
- позволява обекти с различни размери (полиморфизъм)

A** vs A*

A** - масив от указатели	A*
⊕ Бърз swap и resize	⊕ има locality - по-бързо се обхожда колекцията
⊕ може без default конструктор	⊕ елементите са подредени последователно
⊕ без измисна памет, може обект да е nullptr	⊖ харчим много памет
⊕ при полиморфизъм	⊖ размястването на елементи в колекцията е скъпа операция
⊖ няма locality	

Голямата четворка при масив от указатели

- задължително се разписва има ли в класа масив от указатели A^{**}

Rule of Five и Move семантика в C++

lvalue - израз с постоянен адрес в паметта, стои от ляво на =

rvalue - израз без постоянен адрес (временна стойност)

```
int x;  
x = 5;  
↓   ↘  
lvalue rvalue
```

rvalue референция (&&) - обвързва се само с rvalue; означава, че обектът е временен

std::move - преобразува lvalue в rvalue(rvalue)

Move конструктор - "краде" ресурсите на временен обект вместо да ги копира

Move оператор = - оператор за присвояване, който краде ресурсите на временен обект

Rule of Five - при дефинирано едно от пет: деструктор, copy конструктор, copy оператор =, Move конструктор, Move оператор = - дефинират се и останалите четири

Rule of Three - деструктор + copy конструктор + copy оператор =
- при класове с динамична памет

Rule of Five - $\sim T()$, $T(\text{const } T\&)$, $T\& \text{op}=(\text{const } T\&)$, $T(T\&\&)$, $T\& \text{op}=(T\&\&)$
приема rvalue
↓ move constructor ↓ move operator =

- move операциите крадат ресурси, вместо да ги копират

1. Move конструктор - при създаване на нов обект от rvalue
- краде ресурсите на source обекта и го оставя в празно, валидно състояние

$T(T\&\& \text{other})$ noexcept;

↳ rvalue референция. неконстанта - нумриране other

- взимаме указателя на other (не заресяме нова памет)
- other.name = nullptr - нулираме указателя

2. Move оператор = - при присвояване на ^{обект} value на вече съществуващ

T& operator = (T&& other) noexcept;

- освобождаваме собствените си ресурси (на this)
- крадем ресурсите на other
- нулираме other

noexcept при move - noexcept дава разрешение за move
 - задължително при move!
 - без него се прави copy не move

Пример:

```
const String s("Hello");
```

```
String t = std::move(s);
```

→ иска да създаде нов обект с move,
 но нямаме move конструктор за
 const && other
 ⇒ вика copy конструктор